

Aufgabenblatt 8

Übung zu Einführung in die Bildverarbeitung

Hanna Beitz, Malin Haas, Lasse Huber-Saffer, Tim Radke,
Teodora Rogozenco, Jannis Scheff, Christian Wilms
SoSe 2026

Ausgabe: 12. Juni 2026 - **Abgabe bis: 19. Juni 2026, 10:00 per Moodle!**

Lernziele

- Erkennung von Kanten mit einem oder zwei Filterkernen verstehen
- Kantendetektion sinnvoll anwenden können
- Verarbeitung der rohen Ergebnisse der Kantendetektion erproben

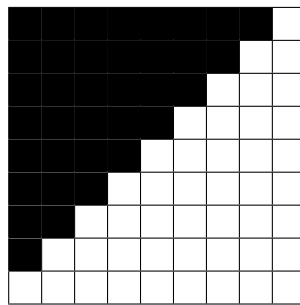


Abbildung 1: Bild mit den Helligkeitswerten 0 (schwarz) und 1 (weiß).

Aufgabe 1 — Eigener Kantendetektor - $10 + 5 + 10 = 25$ Punkte - Theorieaufgabe

1. Entwerft den 3×3 Kernel eines Kantendetektors, der die schräge Kante von weiß zu schwarz in Abbildung 1 bei der Faltung mit dem Bild mit nur diesem einen Kernel erkennt. Dabei soll nur diese Art der schrägen Kante durch den *maximalen* positiven/negativen Ausschlag des Detektors im Ergebnis gekennzeichnet sein. Nutzt als Einträge für euren Kernel Zahlen aus der folgenden Menge: $\{-1, 0, 1\}$.
2. Wie muss euer 3×3 Kernel verändert werden, wenn das Bild in Abbildung 1 invertiert wird, das Ergebnis aber gleich bleiben soll? Nutzt als Einträge für euren Kernel erneut Zahlen aus der folgenden Menge: $\{-1, 0, 1\}$.
3. Können auch die Prewitt-Operatoren die Kante in Abbildung 1 sowie deren Orientierung erkennen? Begründet eure Antwort.

Aufgabe 2 — Farbkanten finden - $5 + 15 + 5 = 25$ Punkte - Programmieraufgabe

Erlaubte (Sub-)Pakete: `numpy`, `skimage.io`, `skimage.filters`, `skimage.color`, `matplotlib`

Die Detektion von Kanten beschränkte sich bisher auf Grauwertbilder. Nun sollen auch Farbinformationen zur Ermittlung bestimmter Kanten, hier der Kante zwischen einem rötlichen See und seiner Umgebung, genutzt werden. Aus diesen Kanten kann in weiteren Schritten die Uferlinie erzeugt werden.

1. Ladet euch zunächst das Bild `lake.png`, das einen rötlichen See inmitten von Wald und Küste zeigt, aus dem Moodle in Python und zeichnet es zunächst mit einem Gauß-Filter und $\sigma = 2$ weich. Wandelt das Bild dann mit der Funktion `skimage.color.rgb2gray()` in ein Grauwertbild um. Ermittelt nun die Gradientenstärke je Pixel über die Sobel-Operatoren und visualisiert das Ergebnis. Welche Kanten werden gefunden und welche sind besonders stark?
2. Da wir nur an der Kante zwischen dem See und der Umgebung interessiert sind, wollen wir nun die Farbinformationen des Bildes nutzen, um die Detektion von Kanten auf jene Kanten zu fokussieren. Dazu soll das weichgezeichnete RGB-Bild zunächst in einen anderen Farbraum umgewandelt werden, der Rot von anderen Farben und Graustufen trennt. Überlegt euch dazu eine Transformationsfunktion, die alle Farben des RGB-Farbraums auf eine Achse abbildet, die etwa die Rotheit kodiert. Implementiert nun die Umwandlung des Bildes. Das Ergebnis ist wiederum ein Graustufenbild, da alle Farbwerte auf eine Achse abgebildet werden sollen, ähnlich der Achse der Graustufen im RGB-Farbraum. Streckt abschließend den Kontrast des Bildes in eurem neuen Farbraum so, dass das Minimum im Bild 0 ist und das Maximum 255.
3. Erzeugt euch nun erneut die Gradientenstärke je Pixel über die Sobel-Operatoren und visualisiert das Ergebnis. Wie unterscheidet es sich vom vorherigen Ergebnis auf Basis der Graustufen?

Aufgabe 3 — Kanten ausdünnen - 10+20+15+5 = 50 Punkte - Programmieraufgabe

Erlaubte (Sub-)Pakete: `numpy`, `skimage.io`, `skimage.filters`, `matplotlib`, `math`

Ein Problem, das bei der bisherigen Kantendetektion auftrat, ist die Breite der Kanten. Die Breite der Kanten ist ein Problem, da Kanten im Bild meist idealerweise 1 Pixel breit sind, in den Ergebnissen der Kantendetektoren jedoch oft durch breitere Bereiche dargestellt werden. Für dieses Problem soll im Rahmen dieser Aufgabe eine Lösung erarbeitet werden, indem alle Pixel deren Gradientenstärke bezüglich einer Nachbarschaft nicht maximal ist, unterdrückt werden (auf 0 setzen). Die Nachbarschaft wird dabei durch die Orientierung der Gradienten bestimmt und verläuft orthogonal zur Kante.

Hinweis: Das Ergebnis der vorherigen Aufgabe, das Bild im Rotheits-Farbraum, findet ihr als CSV-Datei im Moodle. Ladet das Ergebnis mit:

```
rImg = np.loadtxt('ergebnis_Aufgabe2.csv')
```

1. Der Ausgangspunkt dieser Aufgabe ist das Ergebnis von Aufgabe 2, d.h. das Bild der Gradientenstärken auf Basis des neuen Farbraums, der die Blauheit darstellt. Ermittelt zusätzlich zu den Gradientenstärken die Orientierungen (in Grad) der Gradienten je Pixel. Fasst die Orientierungen o in vier Bereiche zu neuen Orientierungen o_{neu} zusammen:

$$o_{neu} = \begin{cases} 0 & \text{falls } -22.5 \leq o < 22.5 \\ 45 & \text{falls } 22.5 \leq o < 67.5 \\ -45 & \text{falls } -67.5 \leq o < -22.5 \\ 90 & \text{sonst.} \end{cases}$$

Das heißt, nach dieser Operation sollen alle Pixel den Wert 0, 45, -45 oder 90 haben. Erzeugt zudem ein Ergebnisbild, das Größe und Datentyp des Bildes der Gradientenstärke haben soll, jedoch mit Nullen gefüllt ist.

2. Nun iteriert durch das Bild und ermittelt zunächst für jedes Pixel jene beiden Nachbarn aus der 3×3 -Nachbarschaft, die orthogonal zur Kante (s. neu ermittelte Gradientenorientierung) zum zentralen Pixel benachbart sind. Achtet dabei auch darauf, dass die Nachbarn im Bild existieren müssen und nicht außerhalb des Bildes liegen dürfen.
3. Ermittelt nun für jedes Pixel, ob die Gradientenstärke des Pixels größer oder gleich ist als jene seiner beiden in der vorherigen Teilaufgabe ermittelten Nachbarn. Ist dies der Fall, so wird für das Ergebnisbild die Gradientenstärke in diesem Pixel übernommen. Sollte dies nicht der Fall sein, wird die Gradientenstärke im Ergebnisbild auf 0 gesetzt. Visualisiert nun das Ergebnis, d.h. das Ergebnisbild mit den ausgedünnten Gradientenstärken. Hatte die Ausdünnung Erfolg?
4. Binariert das Ergebnis der ausgedünnten Gradientenstärken. Wählt einen Grenzwert, sodass ein möglichst durchgehender Kantenzug zwischen See und Umgebung entsteht.

Zusatzaufgabe 4 — Kanten verbinden - 5+20=25 Punkte - Programmieraufgabe

Erlaubte (Sub-)Pakete: `numpy`, `skimage.io`, `skimage.filters`, `matplotlib`, `math`

Aus der vorherigen Aufgabe bleibt das Problem, dass entweder sehr viele für unsere Aufgabe nicht nötige Kanten im Ergebnis verbleiben oder der Kantenzug stark unvollständig ist. Im Rahmen dieser Aufgabe, soll der Kantenzug nun tlw. vervollständigt werden. Nehmt dazu das Ergebnis der ausgedünnten Gradientenstärken der vorherigen Aufgabe ohne finale Binarisierung.

Hinweis: Das Ergebnis der vorherigen Aufgabe, das Bild der ausgedünnten Gradientenstärken findet ihr als CSV-Datei im Moodle. Ladet sie, wie oben beschrieben.

1. Bestimmt zuerst zwei Grenzwerte für die Gradientenstärken. Der eine Grenzwert soll so hoch sein, dass praktisch nur Kantenpixel im Ergebnis verbleiben, die auch wirklich zur gesuchten Kante zwischen Himmel und Gebäude gehören. Der andere Grenzwert soll so niedrig sein, dass praktisch nur noch Kantenpixel im Ergebnis verbleiben, die nicht mehr zur gesuchten Kante gehören.
2. Alle Pixel oberhalb des größeren Grenzwertes gehören schonmal zu unserem finalen Ergebnis, es sind die starken Kanten. Alle Pixel unterhalb des kleineren Grenzwertes gehören in jedem Falle nicht dazu, dies sind die schwachen Kanten. Die restlichen Pixel, deren Gradientenstärken zwischen den Grenzwerten liegt, müssen nun weiter betrachtet werden. Ein solches Pixel soll dabei nur zum finalen Kantenbild gehören, falls es von dem Pixel eine Verbindung zu einer starken Kante gibt. Die Verbindung kann dabei über weitere noch nicht zugeordnete Pixel gehen. Zudem sollen alle acht Nachbarn eines Pixels für eine Verbindung in Frage kommen. Implementiert dies und sucht geeignete Grenzwerte, sodass nur noch die Kante zwischen See und Umgebung übrig bleibt. Zeigt das finale Ergebnis an.